Book 3

Atlas Intelligence & Evaluation Framework

Document Type: AI Evaluation & Benchmark Methodology Specification (AEBMS)

Version: 0.1

Status: Engineering & Research Draft

Classification: Internal Engineering Documentation

Depends On: Book 1 – Atlas Foundation, Book 2 – Architecture & Specifications

Chapter 1

Atlas Evaluation Philosophy

Document Information

Property	Value
Status	Approved
Engineering Phase	Evaluation Science
Priority	Critical
Owner	Evaluation Research Team
Related Books	Book 1, Book 2, Book 5

1.1 Purpose

The purpose of Atlas is not merely to execute benchmarks.

Its primary objective is to evaluate Artificial Intelligence systems in a manner that is:

- Transparent
- Reproducible
- Extensible
- Scientifically defensible

- Practical for engineering teams

Evaluation is therefore treated as a discipline rather than a collection of isolated benchmark scores.

This chapter establishes the philosophical and methodological principles that govern every evaluation performed within Atlas.

1.2 The Atlas Definition of Evaluation

Atlas defines AI evaluation as:

> A structured, reproducible process of measuring the observable capabilities of an intelligent system under controlled conditions using versioned benchmarks, documented methodologies, and transparent reporting.

This definition intentionally avoids claims about intelligence itself.

Atlas measures observable behavior, not consciousness, reasoning mechanisms, or internal model representations.

1.3 Core Principles

Every evaluation performed by Atlas shall satisfy the following principles.

Principle 1 — Reproducibility

An evaluation should produce equivalent results when repeated under identical conditions.

Atlas therefore records:

- Benchmark version
- Model identifier
- Execution configuration
- Adapter version
- Evaluation strategy
- Timestamp
- Software version

Complete reproducibility may not always be achievable for nondeterministic models, but Atlas shall record sufficient metadata to explain variation.

Principle 2 — Transparency

Evaluation methodology must always be visible.

Reports shall document:

- Scoring methodology
- Evaluation strategy
- Benchmark version

- Known limitations
- Confidence level (where applicable)

Atlas rejects "black-box" scoring systems whose methodology cannot be inspected.

Principle 3 — Multi-Dimensional Evaluation

Atlas does not assume intelligence can be represented by a single score.

Instead, evaluations produce multiple dimensions corresponding to different capabilities.

Examples include:

- Coding
- Reasoning
- Mathematical ability
- Tool use
- Instruction following
- Safety
- Factual accuracy
- Consistency

Composite scores, if presented, must always be accompanied by their underlying dimensions.

Principle 4 — Separation of Measurement and Interpretation

Atlas distinguishes between:

- Measurement — objective data produced by benchmark execution.
- Interpretation — conclusions drawn from that data.

For example:

- Measurement: 84% hidden test pass rate.
- Interpretation: Strong coding capability.

Reports shall clearly distinguish measured values from interpretive commentary.

Principle 5 — Benchmark Independence

Evaluation methodology should not depend upon a particular benchmark.

New benchmark categories should integrate without requiring changes to the evaluation framework.

This allows Atlas to evolve as new AI capabilities emerge.

1.4 Evaluation Lifecycle

Every evaluation follows a standardized lifecycle.

Benchmark Selection

↓
Benchmark Version Resolution
↓
Execution Configuration
↓
Model Execution
↓
Raw Output Collection
↓
Evaluation Strategy
↓
Metric Calculation
↓
Capability Profile Generation
↓
Report Generation
↓
Historical Storage

Each stage generates structured metadata.

1.5 Observable Capabilities

Atlas measures observable capabilities rather than latent properties.

Examples include:

Capability	**Example Measurement**
Coding	Hidden test success rate
Reasoning	Correct logical conclusions
Mathematics	Problem-solving accuracy
Tool Use	Successful tool invocation
Planning	Completion of multi-step tasks
Safety	Policy compliance
Consistency	Stability across repeated runs

New capability dimensions may be introduced through benchmark extensions.

1.6 Evaluation Dimensions

Every benchmark declares the dimensions it measures.

A benchmark may measure one or many dimensions.

Example:

Benchmark
↓
Coding
Reasoning
Planning
Tool Use

Evaluation strategies are selected according to these declared dimensions.

1.7 Trust Through Methodology

Atlas does not claim trust because it is the evaluator.

Trust is earned through:

- Open methodology.
- Versioned benchmarks.
- Reproducible execution.
- Transparent reports.
- Independent verification.

The platform itself should never be the sole source of authority.

1.8 Limitations

Atlas explicitly recognizes several limitations of AI evaluation.

- No benchmark measures every capability.
- Models evolve over time.
- Public benchmarks risk contamination.
- Human judgement may vary.
- Automated judges may exhibit bias.

These limitations shall be documented rather than concealed.

1.9 Engineering Principles

Every evaluation subsystem should satisfy:

- Determinism where possible.
- Extensibility.
- Benchmark independence.
- Metric transparency.
- Version awareness.
- Auditability.

1.10 Future Directions

Future research areas include:

- Adaptive benchmark generation.
- Benchmark contamination detection.
- Confidence estimation.
- Human–AI hybrid judging.
- Continuous evaluation pipelines.
- Dynamic benchmark difficulty adjustment.

These remain research topics until validated and are not part of the Version 1 platform.

Builder Notes

Property	Value
Difficulty	Medium
Owner	Evaluation Research Team
Prototype Required	No
Dependencies	Book 2
Research Required	High

Engineering Notes

Confirmed Decisions

- Atlas measures observable behavior rather than attempting to define intelligence.
- Evaluations are multi-dimensional.
- Methodology is transparent and versioned.
- Reports distinguish measured data from interpretation.

Open Questions

- How should confidence be quantified for nondeterministic models?

- Which evaluation dimensions should become mandatory across all benchmark categories?
- When is a composite score appropriate, and how should its weighting be justified?

Chapter 2

Benchmark Lifecycle & Governance

Document Information

Property	Value
Status	Approved
Engineering Phase	Evaluation Framework
Priority	Critical
Owner	Benchmark Research Team
Depends On	Book 1, Book 2
Related Books	Book 4, Book 5

2.1 Purpose

A benchmark is not merely a collection of prompts or test cases.

Within Atlas, a benchmark is treated as a versioned engineering artifact that possesses a lifecycle, metadata, governance, and historical continuity.

The purpose of this chapter is to define how benchmarks are:

- designed,
- validated,
- published,
- executed,
- evolved,
- archived,
- and reproduced.

Every benchmark in Atlas shall follow this lifecycle regardless of its domain.

2.2 Definition of a Benchmark

Atlas defines a benchmark as:

> A structured, version-controlled collection of evaluation tasks, metadata, scoring rules, and execution requirements designed to measure one or more observable capabilities of an AI system.

A benchmark therefore consists of much more than prompts.

Every benchmark contains:

- Metadata
- Tasks
- Evaluation Strategy
- Dataset References
- Version History
- Licensing Information
- Execution Requirements
- Documentation

2.3 Benchmark Lifecycle

Every benchmark progresses through defined stages.

Idea
↓
Design
↓
Draft
↓
Validation
↓
Review
↓
Published
↓
Active Evaluation
↓
Version Update
↓
Archive

A benchmark cannot skip lifecycle stages without explicit administrative approval.

2.4 Stage 1 — Benchmark Proposal

Every benchmark begins as a proposal.

A proposal records:

- Title
- Objective
- Capability being measured
- Motivation
- Intended users
- Expected evaluation methodology

At this stage no executable benchmark exists.

The proposal is a design document.

2.5 Stage 2 — Benchmark Design

The benchmark author defines:

- Problem structure
- Dataset sources
- Input format
- Output format
- Scoring strategy
- Hidden tests
- Constraints
- Required execution environment

The output of this stage is a complete benchmark specification.

2.6 Stage 3 — Draft Implementation

The benchmark is implemented using the Atlas Benchmark Schema.

Required components include:

- Metadata
- Public tasks
- Hidden evaluation tasks
- Configuration
- Evaluation strategy
- Version information

Draft benchmarks are executable but not yet trusted.

2.7 Stage 4 — Validation

Validation ensures the benchmark is technically correct.

Validation includes:

- **Schema Validation:** All required fields exist.

- **Dataset Validation:** Referenced datasets are accessible and correctly licensed.
- **Execution Validation:** The benchmark executes successfully inside the supported environment.
- **Determinism Validation:** Repeated executions produce consistent results where expected.
- **Documentation Validation:** Benchmark documentation is complete.

2.8 Stage 5 — Review

Benchmarks enter a review process before publication.

Review evaluates:

- Technical correctness
- Licensing
- Evaluation methodology
- Clarity
- Reproducibility

Atlas Version 1 uses founder review.

Future versions may introduce community review.

2.9 Stage 6 — Publication

Publishing creates an immutable benchmark version.

Example:

Reasoning Benchmark
↓
Version 1.0.0
↓
Immutable Artifact

Published benchmarks cannot be modified.

Corrections require a new version.

2.10 Stage 7 — Evaluation

Once published, benchmarks become available for execution.

Every execution records:

- Benchmark Version
- Adapter
- Model

- Configuration
- Timestamp
- Results

This ensures future reproducibility.

2.11 Stage 8 — Version Evolution

Benchmarks evolve through semantic versioning.

Examples:

1.0.0
↓
1.1.0
↓
2.0.0

Rules:

- **Major Version:** Breaking evaluation changes.
- **Minor Version:** Additional tasks.
- **Patch Version:** Documentation or metadata fixes.

Historical versions remain executable.

2.12 Stage 9 — Archival

A benchmark may be archived when:

- It becomes obsolete.
- Licensing changes.
- Severe flaws are discovered.
- It is superseded.

Archived benchmarks remain reproducible but are no longer recommended for new evaluations.

2.13 Benchmark Governance

Every benchmark shall have:

- Owner
- Maintainer
- License
- Status
- Version History
- Changelog

Governance ensures accountability throughout the benchmark lifecycle.

2.14 Benchmark Quality Criteria

Before publication, benchmarks should satisfy:

- Clearly defined objective.
- Transparent methodology.
- Reproducible execution.
- Appropriate difficulty.
- Complete documentation.
- Licensing verification.

These criteria establish minimum quality expectations rather than absolute guarantees.

2.15 Benchmark Health

Atlas records health indicators for each benchmark.

Potential indicators include:

- Execution success rate
- Stability across versions
- Maintenance activity
- Reported issues
- Community adoption (future)

These metrics help identify benchmarks that require maintenance.

2.16 Benchmark Categories

Every benchmark belongs to one or more categories.

Examples:

- Coding
- Reasoning
- Mathematics
- Tool Use
- Planning
- Retrieval
- Safety
- Multimodal
- Agentic Systems

A benchmark may measure multiple capability dimensions simultaneously.

2.17 Benchmark Metadata

Minimum metadata includes:

- Identifier
- Name
- Description
- Version
- Author
- License
- Creation Date
- Last Updated
- Difficulty
- Categories
- Evaluation Strategy
- Required Environment

Metadata forms part of the benchmark artifact and is versioned alongside benchmark content.

2.18 Benchmark Traceability

Every benchmark should trace back to:

- Product Goal
- Capability Dimension
- Evaluation Strategy
- Dataset Sources
- Reports Generated

This enables complete lifecycle auditing.

2.19 Future Evolution

Future research may explore:

- Adaptive benchmark generation
- Automatic difficulty calibration
- Benchmark contamination monitoring
- Community governance
- Benchmark recommendation systems

These capabilities are outside the scope of Version 1.

Builder Notes

Property	Value
Difficulty	High
Owner	Benchmark Team

Research Required	High
Prototype Required	Yes
ML Required	No (Version 1)

Engineering Notes

Confirmed Decisions

- Benchmarks are versioned engineering artifacts.
- Publication creates immutable benchmark versions.
- Historical reproducibility is mandatory.
- Governance is part of the benchmark lifecycle.

Open Questions

- How should benchmark quality be quantified?
- When should community governance replace founder review?
- What criteria should trigger benchmark retirement?

Chapter 3

Atlas Benchmark Schema (ABS)

Document Type: Benchmark Specification Standard

Version: ABS v1.0

Document Information

Property	**Value**
Status	Approved
Priority	Critical
Owner	Benchmark Research Team
Standard	Atlas Benchmark Schema v1
Related Books	Book 2, Book 4

3.1 Purpose

Every benchmark in Atlas shall conform to a standardized specification known as the Atlas Benchmark Schema (ABS).

The schema provides a universal representation for benchmarks regardless of domain.

Whether evaluating:

- Coding
- Reasoning
- Mathematics
- Planning
- Agent Systems
- Tool Use
- Safety
- Retrieval
- Vision

...the benchmark shall be represented using the same structural model.

This allows the Evaluation Engine to process every benchmark uniformly.

3.2 Design Goals

The Atlas Benchmark Schema is designed to satisfy the following goals.

- **Universal:** Support every benchmark category.
- **Versioned:** Every benchmark definition is immutable once published.
- **Portable:** A benchmark should execute on any Atlas installation.
- **Extensible:** Future fields may be added without breaking compatibility.
- **Human Readable:** Benchmark authors should understand the structure.
- **Machine Readable:** The Evaluation Engine should validate the schema automatically.

3.3 Benchmark Artifact

A benchmark is not a single file.

It is a structured artifact.

Benchmark Artifact

```
├── metadata.yaml
├── benchmark.json
├── tasks/
├── datasets/
├── hidden_tests/
├── evaluation/
├── documentation/
├── assets/
```

└── changelog.md

Everything required to execute the benchmark lives inside the artifact.

3.4 Metadata Schema

Every benchmark must include metadata.

Required fields:

- benchmark_id:
- name:
- description:
- author:
- organization:
- version:
- license:
- created_at:
- updated_at:
- difficulty:
- estimated_runtime:
- category:
- tags:
- language:
- evaluation_strategy:

These fields are mandatory.

3.5 Capability Declaration

Every benchmark declares which capabilities it measures.

Example:

capabilities:
- coding
- reasoning
- planning
- safety

This allows Atlas to automatically update capability profiles after evaluation.

3.6 Task Definition

A benchmark contains one or more tasks.

Each task contains:

- task_id:
- title:
- description:
- input:
- expected_output:
- constraints:
- time_limit:
- memory_limit:
- visibility:

Visibility values:

- Public
- Hidden

3.7 Dataset Declaration

Benchmarks reference datasets rather than embedding large datasets directly.

Example:

dataset:
name:
version:
license:
checksum:
location:

Atlas validates datasets before execution.

3.8 Evaluation Strategy

Each benchmark specifies its evaluation strategy.

Supported Version 1 strategies:

- Hidden Tests
- Exact Match
- Rule-Based Validation
- Metric Calculation

Future:

- LLM Judge
- Human Judge
- Hybrid Judge

3.9 Execution Requirements

Each benchmark declares:

execution:
python_version:
required_packages:
docker_image:
gpu_required:
network_access:
timeout:

Execution environments become reproducible.

3.10 Resource Limits

Every benchmark declares resource limits.

Examples: CPU, Memory, Execution Time, Disk, Network.

These limits prevent abusive benchmark behavior.

3.11 Hidden Assets

The benchmark artifact separates public and private information.

Benchmark
│
├── Public
│ ├── Description
│ ├── Public Tasks
│ └── Documentation
│
└── Hidden
├── Hidden Tests
├── Evaluation Keys
└── Secret Validation Rules

Only the Evaluation Engine accesses hidden assets.

3.12 Versioning Rules

Atlas follows Semantic Versioning.

- **Major:** Breaking benchmark changes.

- **Minor:** Additional tasks.
- **Patch:** Metadata corrections.

Published versions remain immutable.

3.13 Validation Rules

Before publication, the Benchmark Validator checks:

- [x] Required metadata
- [x] Schema correctness
- [x] Dataset availability
- [x] License validity
- [x] Hidden test integrity
- [x] Execution configuration
- [x] Documentation completeness

Only validated benchmarks may be published.

3.14 Benchmark Packaging

A benchmark package contains:

```
atlas_benchmark.zip
↓
metadata.yaml
benchmark.json
tasks/
datasets/
evaluation/
docs/
```

Packages become portable across Atlas installations.

3.15 Benchmark Manifest

Every package contains a manifest.

Example:

```
schema_version:
benchmark_version:
artifact_hash:
dataset_hashes:
creation_timestamp:
signature:
```

This ensures artifact integrity.

3.16 Integrity Verification

Atlas verifies:

- SHA-256 checksum
- Manifest signature
- Dataset checksums
- Hidden test integrity

Corrupted benchmark artifacts cannot execute.

3.17 Compatibility

Atlas guarantees:

Version 1 Evaluation Engine

↓

All ABS v1 Benchmarks

Future Evaluation Engines

↓

Backward compatibility where practical

3.18 Benchmark Registry

Every benchmark registered in Atlas stores:

- Metadata
- Version
- Capabilities
- Categories
- Dependencies
- Evaluation Strategy
- Health Status

The registry serves as the catalog for all benchmark artifacts.

3.19 Builder Notes

Property	Value

Difficulty	High
Research	Medium
Owner	Benchmark Team
ML Required	No

Engineering Notes

Confirmed Decisions

- Every benchmark conforms to ABS.
- Benchmark artifacts are portable.
- Published versions are immutable.
- Hidden assets remain inaccessible to evaluated models.
- Schema validation is mandatory.

Open Questions

- Should ABS support benchmark inheritance?
- How should multimodal assets be packaged?
- When should benchmark signing become mandatory?

Chapter 4

Capability Taxonomy & Benchmark Categories

Document Type: Capability Classification Specification

Version: CTS v1.0

Document Information

Property	**Value**
Status	Approved
Priority	Critical
Owner	Evaluation Research Team
Standard	Atlas Capability Taxonomy (ACT)
Related Books	Book 2, Book 4

4.1 Purpose

Atlas evaluates capabilities, not models.

A model is simply the subject being evaluated.

The purpose of this chapter is to define the complete taxonomy of measurable AI capabilities supported by Atlas.

Every benchmark, evaluation strategy, metric, and report references this taxonomy.

The taxonomy provides a common language for describing what an AI system can demonstrably do.

4.2 Why a Capability Taxonomy?

Most leaderboards assign a single score or rely on a small number of benchmark names.

Atlas instead evaluates capability dimensions.

Benefits include:

- Better interpretability.
- Easier comparison across benchmarks.
- Extensibility for future AI systems.
- Reduced dependence on any single benchmark.
- Consistent reporting across domains.

A capability taxonomy also allows Atlas to compare models evaluated on different benchmark sets by mapping results to shared capability dimensions.

4.3 Taxonomy Structure

Atlas organizes capabilities into four hierarchical levels.

Domain
↓
Capability
↓
Sub-Capability
↓
Metric

Example:

Reasoning
↓

Logical Reasoning
↓
Deductive Reasoning
↓
Accuracy

This hierarchy enables fine-grained analysis while supporting high-level summaries.

4.4 Primary Capability Domains

Atlas Version 1 defines the following primary capability domains.

Domain	Description
Coding	Software development and debugging
Reasoning	Logical inference and structured thinking
Mathematics	Symbolic and numerical problem solving
Planning	Multi-step task decomposition
Tool Use	Effective use of external tools
Knowledge	Retrieval and factual recall
Safety	Safe and policy-compliant behavior
Language	Communication and language understanding
Vision (Future)	Image understanding and reasoning
Multimodal (Future)	Cross-modal reasoning

These domains represent broad evaluation categories rather than individual benchmarks.

4.5 Coding Domain

Coding evaluates software engineering capability.

Sub-capabilities include:

- Code Generation
- Code Completion

- Bug Fixing
- Refactoring
- Unit Test Generation
- Code Explanation
- Documentation
- API Usage
- Repository Navigation
- Build System Understanding

Example metrics:

- Hidden Test Pass Rate
- Compilation Success
- Static Analysis Warnings
- Execution Time

4.6 Reasoning Domain

Reasoning measures structured cognitive processes.

Sub-capabilities:

- Deductive Reasoning
- Inductive Reasoning
- Abductive Reasoning
- Analogical Reasoning
- Causal Reasoning
- Counterfactual Reasoning

Example metrics:

- Logical Accuracy
- Chain Consistency
- Answer Correctness

4.7 Mathematics Domain

Mathematical capability includes:

- Arithmetic
- Algebra
- Geometry
- Probability
- Statistics
- Calculus
- Symbolic Manipulation

Metrics include:

- Correct Solution Rate

- Step Consistency
- Numerical Precision

4.8 Planning Domain

Planning measures the ability to organize and execute multi-step objectives.

Sub-capabilities:

- Goal Decomposition
- Task Ordering
- Resource Allocation
- Constraint Satisfaction
- Long-Horizon Planning

Metrics:

- Plan Validity
- Task Completion
- Efficiency

4.9 Tool Use Domain

Measures interaction with external systems.

Examples:

- File Operations
- Terminal Usage
- Web Search
- API Calling
- Database Queries
- Function Calling

Metrics:

- Successful Tool Invocations
- Tool Accuracy
- Recovery from Errors

4.10 Knowledge Domain

Measures information retrieval and factual understanding.

Sub-capabilities:

- World Knowledge
- Scientific Knowledge
- Technical Knowledge
- Domain Expertise
- Citation Accuracy

Metrics:

- Factual Accuracy
- Retrieval Precision
- Hallucination Rate

4.11 Safety Domain

Safety evaluates responsible behavior.

Sub-capabilities:

- Harmful Content Refusal
- Privacy Protection
- Bias Mitigation
- Secure Coding
- Prompt Injection Resistance

Metrics:

- Policy Compliance
- Unsafe Response Rate
- Attack Resistance

4.12 Language Domain

Language capability includes:

- Reading Comprehension
- Writing Quality
- Translation
- Summarization
- Conversation
- Instruction Following

Metrics:

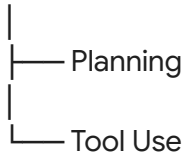
- BLEU (where applicable)
- ROUGE (where applicable)
- Instruction Success
- Human Preference (future)

4.13 Capability Relationships

Capabilities rarely exist in isolation.

Example:

Coding
    │
    ├── Reasoning



A coding benchmark may simultaneously evaluate reasoning and planning.

Atlas therefore records primary and secondary capability mappings for every benchmark.

4.14 Benchmark-to-Capability Mapping

Each benchmark declares:

```
primary_capability:
  - Coding

secondary_capabilities:
  - Planning
  - Tool Use
  - Reasoning
```

The Evaluation Engine uses these mappings when generating capability profiles.

4.15 Capability Evolution

The taxonomy is expected to evolve.

New domains may be introduced through versioned revisions of the Atlas Capability Taxonomy.

Examples:

- Robotics
- Audio
- Scientific Discovery
- Autonomous Agents
- Long-Term Memory

Older benchmarks remain valid because mappings are versioned.

4.16 Taxonomy Governance

Changes to the taxonomy require:

- Research justification.
- Architecture Decision Record (ADR).
- Version increment.

- Migration documentation.

Breaking changes require a new taxonomy version.

4.17 Builder Notes

Property	Value
Difficulty	Medium
Research Required	High
Owner	Evaluation Research Team
ML Required	No

Engineering Notes

Confirmed Decisions

- Atlas evaluates capabilities, not benchmark names.
- Every benchmark maps to one or more capability domains.
- The taxonomy is hierarchical and versioned.
- Capability profiles derive from taxonomy mappings.

Open Questions

- How granular should sub-capabilities become?
- Should community extensions to the taxonomy be allowed?
- How should emerging AI capabilities be incorporated?

Chapter 5

Evaluation Execution Pipeline

Document Type: Evaluation Pipeline Specification

Version: EPS v1.0

Document Information

Property	Value
Status	Approved

Priority	Critical
Owner	Evaluation Engineering Team
Depends On	Chapters 1–4
Related Books	Book 2, Book 4

5.1 Purpose

The Evaluation Execution Pipeline defines the complete lifecycle of an evaluation run.

Its objective is to ensure that every benchmark execution follows a deterministic, auditable, and reproducible workflow regardless of benchmark type or execution backend.

Every evaluation performed by Atlas shall pass through this pipeline.

5.2 Pipeline Objectives

The pipeline must guarantee:

- Reproducibility
- Transparency
- Fault Isolation
- Scalability
- Version Awareness
- Complete Auditability

The pipeline intentionally separates execution from evaluation.

Running a model and scoring its output are independent stages.

5.3 High-Level Pipeline

User Request
↓
Request Validation
↓
Benchmark Resolution
↓
Environment Preparation
↓
Execution Adapter Selection
↓
Model Execution
↓

Raw Output Capture
↓
Evaluation Strategy
↓
Metric Calculation
↓
Capability Mapping
↓
Score Aggregation
↓
Report Generation
↓
Result Storage

Every stage records metadata and produces structured logs.

5.4 Stage 1 — Request Validation

Atlas validates:

- User permissions
- Benchmark availability
- Benchmark version
- Model availability
- Adapter health
- Configuration completeness

Validation failures terminate the pipeline before execution begins.

5.5 Stage 2 — Benchmark Resolution

The Benchmark Engine resolves:

- Benchmark ID
- Published version
- Evaluation strategy
- Dataset references
- Hidden assets
- Execution requirements

The resolved benchmark becomes immutable for the duration of the evaluation.

5.6 Stage 3 — Environment Preparation

The Execution Engine prepares the runtime.

Tasks include:

- Load benchmark package
- Resolve datasets
- Verify checksums
- Prepare execution container
- Apply resource limits
- Configure environment variables

This stage ensures identical execution conditions for repeated evaluations.

5.7 Stage 4 — Adapter Selection

The Execution Engine selects an adapter based on:

- User selection
- Project configuration
- Organization defaults
- Platform defaults

Examples:

- Local Adapter (Ollama)
- API Adapter
- Enterprise Adapter

The selected adapter is recorded in evaluation metadata.

5.8 Stage 5 — Model Execution

The adapter executes benchmark tasks.

For every task Atlas records:

- Task ID
- Start time
- Completion time
- Runtime
- Raw output
- Token usage (if available)
- Exit status

Failures are isolated to the task level whenever possible.

5.9 Stage 6 — Output Collection

Outputs are stored without modification.

Collected artifacts include:

- Generated text

- Code
- Logs
- Tool outputs
- Terminal output
- Files (future)

Raw outputs remain immutable.

5.10 Stage 7 — Evaluation Strategy

The Evaluation Engine selects the strategy defined by the benchmark.

Version 1 supports:

- Hidden Test Evaluation
- Exact Match
- Rule-Based Validation
- Metric-Based Scoring

Future strategies:

- LLM-as-Judge
- Human Review
- Hybrid Evaluation

The strategy itself is versioned and recorded.

5.11 Stage 8 — Metric Calculation

Metrics are calculated independently.

Examples:

Capability	**Example Metrics**
Coding	Hidden Test Pass Rate, Compilation Success
Reasoning	Logical Accuracy
Mathematics	Correct Solution Rate
Safety	Policy Compliance
Tool Use	Successful Tool Calls

Each metric includes:

- Name
- Value
- Unit
- Weight
- Calculation method

5.12 Stage 9 — Capability Mapping

Metric results are mapped onto the Atlas Capability Taxonomy.

Example:

Hidden Test Pass Rate
↓
Code Generation
↓
Coding Domain

This enables capability-centric reporting rather than benchmark-centric reporting.

5.13 Stage 10 — Score Aggregation

Atlas aggregates metrics into structured outputs.

Outputs include:

- Raw metrics
- Capability scores
- Benchmark score
- Confidence indicators (future)

Composite scores are optional and never replace detailed metrics.

5.14 Stage 11 — Report Generation

The Reporting Engine creates:

- Summary report
- Detailed report
- Capability profile
- Historical comparison (if applicable)

Each report includes methodology and benchmark version information.

5.15 Stage 12 — Result Storage

Persistent records include:

- Execution metadata

- Evaluation metrics
- Capability profile
- Generated report
- Audit logs

Stored results become immutable historical records.

5.16 Pipeline Metadata

Every evaluation stores:

- Evaluation ID
- Correlation ID
- Benchmark Version
- Adapter Version
- Evaluation Strategy Version
- Timestamp
- Software Version
- Environment Hash

This metadata supports reproducibility and debugging.

5.17 Failure Handling

Failures are categorized as:

- Validation Failure
- Environment Failure
- Adapter Failure
- Model Failure
- Evaluation Failure
- Reporting Failure

Every failure records:

- Error code
- Stage
- Root cause
- Recovery suggestion

5.18 Performance Considerations

Pipeline performance is monitored using:

- Queue time
- Execution time
- Evaluation time
- Report generation time
- Total wall-clock duration

Performance data is stored separately from evaluation scores.

5.19 Pipeline Extensibility

The pipeline is designed to accommodate future additions such as:

- Distributed execution
- Parallel benchmark execution
- Continuous evaluation
- Streaming outputs
- Human review stages
- Adaptive evaluation

New stages may be inserted without redesigning existing components.

5.20 Builder Notes

Property	Value
Difficulty	High
Owner	Evaluation Engineering Team
Prototype Required	Yes
ML Required	No (Version 1)
Research Required	Medium

Engineering Notes

Confirmed Decisions

- Execution and evaluation are separate stages.
- Raw outputs are immutable.
- Every pipeline stage records metadata.
- Evaluation strategies are versioned.
- Capability mapping occurs after metric calculation.

Open Questions

- How should streaming model outputs be evaluated?
- When should distributed execution be introduced?
- Which future stages require human involvement?

Chapter 6

Evaluation Methodology

Document Type: Scientific Evaluation Methodology Specification (SEMS)

Version: SEMS v1.0

Document Information

Property	Value
Status	Approved
Priority	Critical
Owner	Atlas Evaluation Research Team
Standard	Atlas Evaluation Methodology (AEM)
Depends On	Chapters 1–5

6.1 Purpose

Evaluation is the scientific process through which Atlas transforms raw model outputs into objective measurements of capability.

This chapter defines the methodology used by Atlas to ensure evaluations are:

- Fair
- Transparent
- Reproducible
- Extensible
- Scientifically Defensible

Rather than defining a single evaluation algorithm, Atlas defines a methodological framework capable of supporting multiple evaluation strategies while maintaining consistent reporting standards.

6.2 Evaluation Philosophy

Atlas evaluates observable performance, not intelligence itself.

An evaluation is composed of four independent stages:

Observation
↓
Measurement

↓
Interpretation
↓
Reporting

These stages remain distinct to reduce methodological bias.

6.3 Evaluation Principles

Every evaluation performed by Atlas must satisfy the following principles.

Principle 1 — Fairness

Every evaluated model receives identical:

- Benchmark version
- Execution conditions
- Evaluation strategy
- Resource limits

Principle 2 — Transparency

Every score must be explainable.

Atlas never produces unexplained scores.

Every report documents:

- Metrics
- Weighting
- Evaluation strategy
- Benchmark version

Principle 3 — Repeatability

Repeated evaluations should produce equivalent results whenever deterministic execution is possible.

Where deterministic execution is impossible, Atlas records the variance rather than hiding it.

Principle 4 — Separation of Concerns

Execution ≠ Evaluation ≠ Reporting

These remain independent systems.

Principle 5 — Benchmark Independence

Evaluation methodology must work regardless of benchmark category.

6.4 Evaluation Inputs

Every evaluation consumes:

Benchmark
↓
Model Output
↓
Ground Truth
↓
Evaluation Strategy
↓
Metadata

Nothing else influences scoring.

Hidden evaluation assets remain inaccessible to evaluated models.

6.5 Evaluation Strategies

Version 1 supports four evaluation strategies.

Strategy	**Suitable For**
Hidden Test Evaluation	Coding
Exact Match	Mathematics
Rule-Based Validation	Structured outputs
Metric-Based Evaluation	Quantitative tasks

Future strategies:

- LLM-as-Judge
- Human Evaluation
- Hybrid Evaluation
- Tournament Evaluation

Each strategy is versioned independently.

6.6 Metric System

Metrics are atomic measurements.

Examples include:

Capability	Metric
Coding	Hidden Test Pass Rate
Coding	Compilation Success
Reasoning	Logical Accuracy
Mathematics	Numerical Precision
Safety	Policy Compliance
Tool Use	Successful Tool Invocation Rate

Every metric contains:

- Identifier
- Name
- Description
- Unit
- Calculation Method
- Range
- Interpretation Guidance

6.7 Metric Validation

Before a metric is accepted:

- Formula must be documented.
- Inputs must be defined.
- Units must be specified.
- Validation tests must pass.

Metrics become versioned artifacts.

6.8 Multi-Dimensional Evaluation

Atlas intentionally avoids reducing evaluation to a single number.

Each evaluation produces a vector of capability scores.

Example:

Coding 91.4
Reasoning 84.2

Planning	79.1
Tool Use	88.0
Safety	94.5

This preserves nuance and enables meaningful comparison across models.

6.9 Composite Scores

Composite scores may be generated for convenience but are optional.

If a composite score is reported, Atlas must disclose:

- Included metrics
- Weighting methodology
- Normalization method
- Limitations

Composite scores shall never replace underlying capability measurements.

6.10 Evaluation Confidence

Future versions may estimate confidence in evaluation results.

Potential contributors include:

- Number of benchmark tasks
- Result consistency
- Metric stability
- Evaluation strategy maturity

Version 1 reports methodology rather than confidence estimates.

6.11 Variance & Repeatability

For nondeterministic models, Atlas may execute multiple evaluation runs.

Statistics recorded include:

- Mean
- Median
- Standard Deviation
- Minimum
- Maximum

This enables users to understand performance variability rather than relying on a single measurement.

6.12 Normalization

Different metrics use different scales.

Atlas normalizes metrics before aggregation.

Examples:

- Percentage scores
- Binary outcomes
- Time measurements
- Resource usage

Normalization rules are versioned and documented.

6.13 Benchmark Comparability

Two benchmark scores may only be compared if:

- Benchmark versions are compatible.
- Evaluation strategy is compatible.
- Capability mappings overlap.
- Methodology is documented.

Atlas prevents invalid comparisons whenever possible.

6.14 Evaluation Provenance

Every evaluation stores complete provenance.

Required metadata:

- Benchmark Version
- Dataset Version
- Adapter Version
- Evaluation Strategy Version
- Runtime Environment
- Software Version

This ensures historical reproducibility.

6.15 Quality Assurance

Every evaluation strategy undergoes:

- Unit Testing
- Integration Testing
- Regression Testing
- Cross-version Validation

Changes require version increments.

6.16 Methodology Evolution

Evaluation methodology evolves through semantic versioning.

Example:

AEM v1.0
↓
AEM v1.1
↓
AEM v2.0

Major revisions require migration documentation.

6.17 Builder Notes

Property	Value
Difficulty	High
Owner	Evaluation Research Team
Research Required	High
ML Required	No (Version 1)

Engineering Notes

Confirmed Decisions

- Evaluation focuses on observable behavior.
- Metrics are atomic, versioned artifacts.
- Composite scores are optional and transparent.
- Methodology is independent of benchmark type.
- Provenance is mandatory.

Open Questions

- When should confidence estimation be introduced?
- Which normalization techniques are most appropriate across heterogeneous benchmarks?
- How should future LLM-as-Judge outputs be calibrated against objective metrics?

Chapter 7

Capability Profile Engine (CPE)

Document Type: Capability Modeling Specification

Version: CPE v1.0

Document Information

Property	Value
Status	Approved
Priority	Critical
Owner	Evaluation Intelligence Team
Standard	Atlas Capability Profile (ACP)
Depends On	Chapters 1–6

7.1 Purpose

Benchmark scores are isolated observations.

Capability Profiles transform those observations into a structured representation of an AI system.

Rather than asking:

"Did Model X score 91%?"

Atlas asks:

"What capabilities has Model X demonstrated, how confidently, and under which evaluation conditions?"

The Capability Profile becomes the primary output of Atlas.

7.2 Why Capability Profiles?

Traditional leaderboards compare benchmark scores.

Atlas compares capability vectors.

Advantages include:

- Better interpretability.
- Cross-benchmark comparison.
- Historical tracking.
- Capability gap analysis.

- Enterprise-friendly reporting.
- Future ML analytics.

7.3 Capability Profile Structure

A Capability Profile consists of four layers.

Raw Metrics
↓
Capability Scores
↓
Domain Scores
↓
Capability Profile

Each layer preserves traceability back to the original benchmark results.

7.4 Capability Vector

Every evaluated model is represented as a multidimensional vector.

Example:

Coding 91.4
Reasoning 87.2
Planning 79.3
Tool Use 88.6
Safety 95.1
Mathematics 84.5
Knowledge 82.9

This vector is the canonical representation of model performance within Atlas.

7.5 Capability Aggregation

Capability scores are derived from multiple benchmarks.

Example:

HumanEval
|
MBPP
|
SWE-Bench
|

RepoBench
▼
Coding Capability

Atlas therefore measures capabilities rather than individual benchmark performance.

7.6 Primary Capability Model

Each capability consists of:

- Capability ID
- Name
- Description
- Parent Domain
- Supporting Benchmarks
- Supporting Metrics
- Confidence (future)
- Version

Example:

capability_id: CAP-CODE-001
name: Code Generation
domain: Coding
version: 1.0

7.7 Capability Relationships

Capabilities form a graph.

Example:

Reasoning
├── Logical
├── Mathematical
├── Causal
└── Planning

Graph relationships support:

- Similarity analysis.
- Dependency analysis.
- Recommendation systems.
- Future ML research.

7.8 Capability Profile Generation

Generation process:

Benchmark Results
↓
Metric Registry
↓
Capability Mapping
↓
Normalization
↓
Aggregation
↓
Capability Vector
↓
Profile Generation

Every transformation is versioned and auditable.

7.9 Capability Categories

Atlas Version 1 defines:

- **Technical:** Coding, Debugging, Testing, API Usage
- **Cognitive:** Logical Reasoning, Mathematical Reasoning, Planning, Decision Making
- **Knowledge:** Scientific, Technical, General Knowledge
- **Interaction:** Tool Use, Instruction Following, Conversation
- **Safety:** Refusal, Privacy, Robustness, Secure Coding

7.10 Capability History

Profiles evolve over time.

Atlas stores historical snapshots.

Example:

Model X
Version 1
↓
Version 2
↓
Version 3
↓
Version 4

Users can observe how capabilities improve or regress across model releases.

7.11 Capability Drift

Atlas measures changes between evaluations.

Potential indicators:

- Score improvement
- Score degradation
- Capability emergence
- Capability loss

Drift analysis enables longitudinal tracking of model evolution.

7.12 Capability Similarity

Atlas computes similarity between capability vectors.

Potential applications:

- Model recommendation.
- Competitor analysis.
- Cluster discovery.
- Transfer learning research.

Version 1 stores vectors; advanced similarity analysis is introduced in later versions.

7.13 Capability Visualization

Capability Profiles may be visualized using:

- Radar charts
- Heat maps
- Capability trees
- Trend lines
- Domain scorecards

Visualizations are generated from the underlying profile and never replace the raw data.

7.14 Profile Versioning

Every Capability Profile includes:

- Profile Version
- Taxonomy Version
- Metric Registry Version
- Evaluation Methodology Version

This ensures profiles remain reproducible even as Atlas evolves.

7.15 Builder Notes

Property	Value
Difficulty	High
Owner	Evaluation Intelligence Team
Prototype Required	Yes
ML Required	No (Version 1)
Research Required	High

Engineering Notes

Confirmed Decisions

- Capability Profiles are the primary output of Atlas.
- Profiles aggregate benchmark results into capability vectors.
- Every transformation is versioned and traceable.
- Historical profiles are retained for longitudinal analysis.

Open Questions

- How should confidence values be represented?
- What aggregation techniques best preserve interpretability?
- How should future multimodal capabilities integrate into existing profiles?

Atlas Capability Genome (ACG)

Every evaluated model receives a persistent Capability Genome.

Unlike a leaderboard score, the Capability Genome is a structured fingerprint composed of:

- Capability Vector
- Benchmark Coverage
- Metric Distribution
- Capability Strengths
- Capability Weaknesses
- Historical Evolution
- Evaluation Provenance
- Taxonomy Version
- Methodology Version

Conceptually:

```
Capability Genome
├── Coding
│   ├── Generation
│   ├── Debugging
│   └── Testing
│
├── Reasoning
│   ├── Logical
│   ├── Mathematical
│   └── Planning
│
├── Safety
│
├── Knowledge
│
└── Tool Use
```

This genome becomes the canonical identity of a model within Atlas.

Instead of saying "Model A scored 89.4", Atlas can say:

"Model A demonstrates exceptional code generation and secure coding, moderate planning ability, emerging autonomous tool use, and stable reasoning across four benchmark generations."

That transition—from score-centric evaluation to capability-centric intelligence profiling—is, in my view, the strongest long-term differentiator Atlas can build.

Chapter 8

Judge Framework & Evaluation Strategies

Document Type: Judge Architecture Specification

Version: JF v1.0

Document Information

Property	Value
Status	Approved

Priority	Critical
Owner	Evaluation Research Team
Standard	Atlas Judge Framework (AJF)
Depends On	Chapters 1–7

8.1 Purpose

The Judge Framework determines how Atlas evaluates benchmark outputs.

Rather than embedding evaluation logic inside benchmarks, Atlas separates judging into modular strategies.

This architecture enables different benchmark categories to use different evaluation methods while maintaining consistent reporting.

8.2 Design Philosophy

No single judging strategy is appropriate for every benchmark.

Examples:

- Coding ↓ Hidden Tests
- Math ↓ Exact Match
- Planning ↓ Rule-Based Evaluation
- Safety ↓ Policy Evaluation
- Future ↓ LLM-assisted Review

Atlas therefore supports multiple judge types under one framework.

8.3 Judge Hierarchy

Atlas Judge Framework
│
├── Objective Judges
│ ├── Hidden Tests
│ ├── Exact Match
│ ├── Rule Engine
│ └── Metric Validator
│
├── Assisted Judges
│ ├── LLM Judge
│ ├── Human Judge
│ └── Hybrid Judge

```
│
└── Research Judges
├── Consensus Judge
├── Committee Judge
└── Adaptive Judge
```

Each judge implements the same evaluation interface.

8.4 Judge Selection

Each benchmark specifies its preferred judge.

Example:

```
judge:
type: hidden_tests
version: 1.0
```

The Evaluation Engine automatically selects the corresponding implementation.

8.5 Objective Judges

Objective judges produce deterministic results.

Examples:

- Hidden Tests
- Exact Match
- Static Analysis
- Unit Tests
- Syntax Validation

Advantages:

- Fast
- Reproducible
- Transparent

Limitations:

- Limited to objective tasks

8.6 Rule-Based Judge

Suitable for structured outputs.

Example rules:

- JSON schema validation

- Required fields
- Output formatting
- Constraint satisfaction

Used for:

- Planning
- Structured reasoning
- API outputs

8.7 Metric-Based Judge

Evaluates quantitative metrics.

Examples:

- BLEU
- ROUGE
- Precision
- Recall
- F1
- Pass Rate

Every metric references the Atlas Metric Registry.

8.8 Human Judge (Future)

Some benchmark categories cannot yet be fully automated.

Human review may be required for:

- Creativity
- UX
- Writing
- Design
- Long-form planning

Atlas records:

- Reviewer ID
- Evaluation criteria
- Timestamp
- Review notes

Human evaluations are always versioned.

8.9 LLM Judge (Future)

Atlas deliberately treats LLM-based judging as an optional strategy.

Potential use cases:

- Open-ended reasoning
- Research summaries
- Essay evaluation
- Conversational quality

Safeguards:

- Prompt versioning
- Judge model version
- Evaluation prompt archive
- Multiple runs (optional)

LLM judges supplement—not replace—objective metrics where available.

8.10 Hybrid Judge

Hybrid evaluation combines multiple evidence sources.

Example:

Hidden Tests
    |
Rule Engine
    |
Static Analysis
    |
Human Review
    |
LLM Judge
    ▼
Final Evaluation

Each component contributes independently.

8.11 Judge Versioning

Every judge has:

- Judge ID
- Version
- Methodology
- Supported Benchmark Types
- Validation Status

Example:

Hidden Test Judge

↓
Version 1.2

8.12 Judge Registry

Atlas maintains a registry of all judges.

Stored information:

- Judge Name
- Version
- Supported Categories
- Inputs
- Outputs
- Validation Status
- Documentation

The registry enables reproducible evaluations.

8.13 Judge Validation

Before a judge becomes available it must pass:

- Unit Tests
- Regression Tests
- Consistency Tests
- Benchmark Validation
- Performance Tests

No judge may be used in production without validation.

8.14 Judge Limitations

Atlas explicitly documents limitations.

Examples:

- Hidden tests cannot evaluate creativity.
- Exact match may penalize equivalent answers.
- Human reviewers introduce subjectivity.
- LLM judges may inherit model bias.

These limitations are disclosed in evaluation reports.

8.15 Future Judge Research

Research directions include:

- Multi-agent judging
- Consensus judging

- Confidence estimation
- Explainable evaluation
- Active review selection
- Judge disagreement analysis

These remain research topics until validated.

Builder Notes

Property	Value
Difficulty	High
Research Required	High
Prototype Required	Partial
ML Required	No (Version 1)

Engineering Notes

Confirmed Decisions

- Judge logic is separated from benchmarks.
- Objective judges are preferred whenever possible.
- Every judge is versioned.
- Judge limitations are documented.
- LLM judges are optional and future-facing.

Open Questions

- How should disagreement between multiple judges be resolved?
- When is human review required?
- How should future confidence estimation integrate into judge outputs?

Chapter 9

Dataset Strategy & Benchmark Repository

Document Type: Dataset Governance & Benchmark Repository Specification

Version: DGS v1.0

Document Information

Property	Value
Status	Approved
Priority	Critical
Owner	Dataset & Benchmark Team
Standard	Atlas Dataset Framework (ADF)
Depends On	Chapters 1–8

9.1 Purpose

Every benchmark depends upon data.

Datasets determine:

- Task quality
- Evaluation fairness
- Reproducibility
- Scientific validity
- Legal compliance

Atlas therefore treats datasets as first-class engineering assets rather than simple files.

9.2 Philosophy

Atlas never evaluates directly from arbitrary datasets.

Instead:

Dataset
↓
Dataset Registry
↓
Benchmark
↓
Evaluation

Datasets are independent from benchmarks.

Multiple benchmarks may reuse the same dataset.

9.3 Dataset Lifecycle

Every dataset follows a controlled lifecycle.

Proposal
↓
Collection
↓
Cleaning
↓
Validation
↓
Licensing Review
↓
Registration
↓
Versioning
↓
Publication
↓
Archive

Datasets cannot bypass validation.

9.4 Dataset Categories

Atlas Version 1 supports the following dataset classes.

Category	Examples
Coding	HumanEval, MBPP, SWE-bench, RepoBench
Mathematics	GSM8K, MATH
Reasoning	ARC, BIG-Bench
Knowledge	MMLU, GPQA
Tool Use	Terminal-Bench, ToolBench
Safety	AdvBench, XSTest
Agentic	GAIA, WebArena
Multimodal (Future)	MMMU, MMMU-Pro

A benchmark may reference multiple datasets simultaneously.

9.5 Atlas Dataset Registry (ADR)

Every dataset receives a permanent registry entry.

Example:

dataset_id: DATA-CODE-001
name: HumanEval
version: 1.0
license: MIT
checksum: SHA256
owner: OpenAI
category: Coding

The registry becomes the authoritative catalogue of all datasets used within Atlas.

9.6 Dataset Metadata

Every registered dataset contains:

- Dataset ID
- Name
- Version
- Author
- Organization
- License
- Citation
- Download Source
- File Format
- Size
- Language
- Capability Mapping
- Creation Date
- Last Validation Date

Metadata is immutable once published.

9.7 Licensing Policy

Atlas never assumes a dataset may be used.

Every dataset must include:

- License type

- Commercial usage status
- Redistribution status
- Citation requirements
- Modification permissions

Datasets without clear licensing cannot be promoted to "Verified" status.

9.8 Dataset Validation

Validation includes:

- **Integrity:** Checksum verification, File completeness, Corruption detection
- **Quality:** Missing values, Duplicate entries, Invalid records
- **Compatibility:** Schema validation, Benchmark compatibility, Encoding verification

9.9 Dataset Versioning

Datasets use Semantic Versioning.

HumanEval
↓
1.0.0
↓
1.1.0
↓
2.0.0

Historical benchmark executions always reference the exact dataset version used.

9.10 Dataset Provenance

Atlas records the complete origin of every dataset.

Required fields:

- Original source
- Download URL
- Publication
- Authors
- DOI (if available)
- License
- Validation timestamp

This ensures scientific traceability.

9.11 Dataset Relationships

Datasets may support multiple benchmarks.

Example:

```
HumanEval
│
├── Coding Benchmark A
├── Coding Benchmark B
└── Capability Validation Suite
```

Atlas avoids unnecessary duplication.

9.12 Dataset Quality Metrics

Atlas records quality indicators.

Examples:

- Completeness
- Diversity
- Class Balance
- Duplicate Rate
- Validation Status
- Community Trust (future)

These metrics describe the dataset, not model performance.

9.13 Benchmark Repository

The Atlas Benchmark Repository stores:

- Benchmark packages
- Benchmark versions
- Metadata
- Documentation
- Evaluation strategies
- Dataset dependencies

Every published benchmark references registry datasets.

9.14 Benchmark Discovery

Benchmarks become searchable using:

- Capability
- Category
- Difficulty
- Language

- Dataset
- License
- Version

Future releases may include semantic search and recommendation systems.

9.15 Benchmark Collections

Atlas supports benchmark collections.

Example:

Coding Collection
├── HumanEval
├── MBPP
├── SWE-Bench
├── RepoBench
└── LiveCode

Collections simplify large-scale evaluations.

9.16 Dataset Health

Atlas continuously tracks:

- Validation status
- License changes
- Dataset availability
- Integrity checks
- Community issues

Datasets with unresolved issues may be marked as Deprecated or Restricted.

9.17 Future Dataset Strategy

Potential future capabilities include:

- Synthetic benchmark generation
- Automatically curated benchmark sets
- Community dataset submissions
- Federated dataset registries
- Contamination monitoring
- Dataset lineage visualization

These are outside the scope of Version 1.

Builder Notes

Property	Value
Difficulty	High
Owner	Dataset Team
Prototype Required	Yes
ML Required	No (Version 1)
Research Required	Medium

Engineering Notes

Confirmed Decisions

- Datasets are independent, versioned assets.
- Every benchmark references datasets through the Atlas Dataset Registry.
- Licensing validation is mandatory.
- Dataset provenance is recorded.
- Historical reproducibility depends on exact dataset versions.

Open Questions

- Should Atlas mirror datasets or always reference external sources?
- How should restricted or proprietary datasets be represented?
- When should community dataset contributions be enabled?

Chapter 10

Atlas Intelligence Modules & Machine Learning Roadmap

Document Type: Intelligence Architecture Specification

Version: AIM v1.0

Document Information

Property	Value
Status	Approved
Priority	Critical

Owner	AI Research Team
Depends On	Chapters 1–9
Related Books	Book 4, Book 5

10.1 Purpose

Atlas Version 1 does not require machine learning to function.

Instead, it relies on deterministic evaluation methods such as:

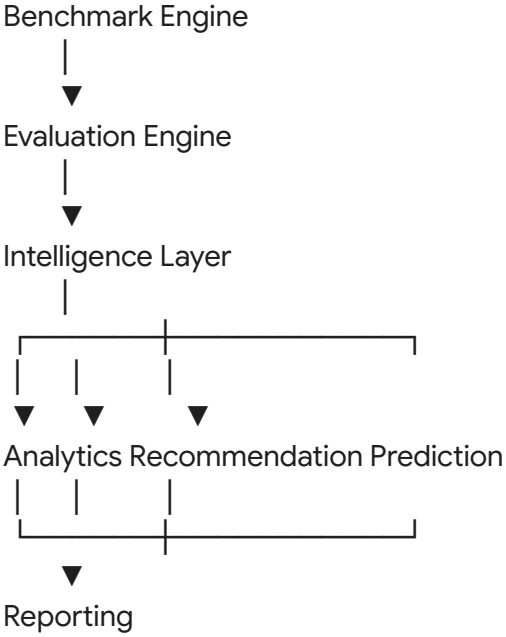
- Hidden tests
- Exact matching
- Rule engines
- Static analysis
- Metric calculation

Machine learning becomes valuable when Atlas evolves beyond deterministic evaluation into adaptive benchmarking, confidence estimation, recommendation systems, and benchmark intelligence.

This chapter defines the intelligence modules that will progressively transform Atlas into an intelligent evaluation platform.

10.2 Intelligence Architecture

Atlas separates intelligence into independent modules.



The Intelligence Layer never executes benchmarks directly.

It analyzes benchmark data after evaluation.

10.3 Version 1 Intelligence

Version 1 contains no trained ML models.

Instead it uses:

- Rule engines
- Statistical analysis
- Capability aggregation
- Trend calculation
- Historical comparison

This minimizes complexity while establishing a clean foundation.

10.4 Intelligence Module Registry

Atlas defines intelligence modules as independent services.

Module	Version 1	Future ML
Capability Aggregator	Rule-Based	ML Enhanced
Benchmark Recommendation	Rule-Based	Graph ML
Difficulty Estimator	Manual	Regression Model
Similarity Engine	Statistical	Embedding Model
Trend Analyzer	Statistical	Forecasting
Confidence Estimator	N/A	ML
Benchmark Mutation	N/A	Generative AI

Each module evolves independently.

10.5 Capability Aggregation Engine

Purpose: Transform benchmark metrics into capability profiles.

Inputs:

- Metric Registry
- Capability Taxonomy
- Benchmark Results

Outputs:

- Capability Vector
- Domain Scores
- Capability Genome

Version 1 uses deterministic aggregation.

No ML required.

10.6 Similarity Engine

Purpose: Compare evaluated models.

Example:

Model A
↓
Capability Vector
↓
Similarity Engine
↓
Top 10 Similar Models

- **Version 1:** Cosine Similarity, Euclidean Distance, Pearson Correlation
- **Future:** Embedding-based similarity, Graph Neural Networks

10.7 Recommendation Engine

Purpose: Recommend benchmarks.

Example:

Model
↓
Capability Gaps
↓
Recommendation Engine
↓
Recommended Benchmarks

- **Version 1:** Rule-based.
- **Future:** Learning-to-Rank. Graph Recommendation.

10.8 Benchmark Difficulty Estimator

Purpose: Estimate benchmark complexity.

- **Version 1:** Manual difficulty assignment.
- **Future ML:** Train regression models using: Historical pass rates, Runtime, Human labels, Benchmark size.
- **Possible algorithms:** XGBoost, Random Forest, LightGBM

10.9 Confidence Estimation Engine

Purpose: Estimate confidence in evaluation results.

Potential inputs: Number of tasks, Variance, Metric agreement, Judge agreement, Historical stability.

- **Future ML:** Calibration models, Bayesian estimation, Uncertainty prediction.
- **Version 1:** Not implemented.

10.10 Trend Analysis Engine

Purpose: Analyze model evolution.

Example:

GPT Version History
↓
Capability Profiles
↓
Trend Analysis
↓
Performance Changes

- **Version 1:** Statistical analysis.
- **Future:** Time-series forecasting. LSTM. Temporal Transformers.

10.11 Benchmark Mutation Engine (Research)

Purpose: Generate new benchmark variants.

Potential applications: Reduce benchmark contamination, Increase diversity, Adaptive evaluation.

Potential techniques: Template mutation, Program synthesis, LLM-assisted generation, Constraint solving.

- **Status:** Research only. Not part of Version 1.

10.12 Hallucination Analysis Engine

Purpose: Analyze factual reliability.

Possible metrics: Citation accuracy, Unsupported claims, Contradictions, Fabricated entities.

- **Version 1:** Rule-based where possible.
- **Future:** Hybrid evaluation. Knowledge graph verification.

10.13 Capability Prediction Engine

Purpose: Estimate likely strengths and weaknesses before full evaluation.

Potential inputs: Existing capability profiles, Similarity graph, Historical evaluations.

Future algorithms: Graph Neural Networks, Gradient Boosting, Representation Learning.

- **Research status:** Exploratory.

10.14 Intelligence Data Sources

Future ML modules may use:

- Evaluation history
- Capability profiles
- Metric registry
- Benchmark metadata
- Execution logs
- User interactions (opt-in)
- Benchmark relationships

No proprietary model outputs are retained beyond user-configured policies.

10.15 Research Pipeline

Every proposed ML module follows:

Research Question
↓
Literature Review
↓
Prototype
↓
Offline Evaluation
↓
Internal Validation
↓
Version 1 Candidate
↓

Production Release

No module enters production without measurable benefit.

10.16 Builder Notes

Property	Value
Difficulty	Very High
Owner	AI Research Team
Version 1 ML Models	None Required
Future Research	Extensive

Engineering Notes

Confirmed Decisions

- Atlas Version 1 is primarily rule-based.
- Intelligence modules are independent services.
- ML is introduced only where it demonstrably improves functionality.
- Every ML component follows a documented research pipeline.

Open Questions

- Which module should receive ML capabilities first?
- What datasets are sufficient for training recommendation or prediction systems?
- How should model performance be validated before deployment?

Chapter 10

Reporting Framework & Evaluation Reports

Document Type: Reporting Framework Specification

Version: ARF v1.0

Document Information

Property	Value
Status	Approved

Priority	Critical
Owner	Reporting & Evaluation Team
Standard	Atlas Reporting Framework (ARF)
Depends On	Chapters 1–9
Related Books	Book 2, Book 4

10.1 Purpose

The final product of every Atlas evaluation is not a score.

It is an evidence-based report.

The Atlas Reporting Framework defines how evaluation results are organized, presented, interpreted, and exported.

Reports must communicate evaluation outcomes without exaggerating model capabilities.

Every report generated by Atlas shall be reproducible, traceable, and methodology-aware.

10.2 Design Principles

Every report shall satisfy the following principles.

- **Principle 1:** Evidence before interpretation.
- **Principle 2:** Every claim must reference measurable metrics.
- **Principle 3:** Every report documents methodology.
- **Principle 4:** Reports remain reproducible.
- **Principle 5:** Reports are versioned.

10.3 Report Generation Pipeline

Evaluation Complete
↓
Collect Metrics
↓
Capability Aggregation
↓
Generate Report Sections
↓
Attach Methodology
↓
Attach Metadata

↓
Generate PDF / JSON
↓
Archive Report

10.4 Report Types

Atlas Version 1 supports multiple report formats.

Report	Purpose
Summary Report	Quick evaluation overview
Detailed Report	Complete benchmark analysis
Capability Report	Capability profile
Comparison Report	Compare multiple models
Benchmark Report	Benchmark-specific statistics
Execution Report	Technical execution logs

10.5 Standard Report Structure

Every report follows the same structure.

Cover Page
↓
Executive Summary
↓
Evaluation Overview
↓
Benchmark Information
↓
Execution Environment
↓
Capability Results
↓
Metric Tables
↓
Evidence
↓

Limitations
↓
Methodology
↓
Appendix

10.6 Executive Summary

The first page contains:

- Evaluation ID
- Model
- Benchmark
- Date
- Overall Summary
- Key Strengths
- Key Weaknesses

This section intentionally avoids technical details.

10.7 Benchmark Information

Every report documents:

- Benchmark Name
- Benchmark Version
- Capability Domains
- Dataset Versions
- Evaluation Strategy
- Judge Version

This information is mandatory for reproducibility.

10.8 Capability Results

Capability results are grouped by domain.

Example:

Domain	**Score**
Coding	91.4
Reasoning	84.2
Planning	80.3

Tool Use	88.5
Safety	95.0

Each score links back to supporting metrics.

10.9 Metric Tables

Every capability expands into metric tables.

Example:

Metric	**Value**
Hidden Test Pass Rate	94%
Compilation Success	100%
Runtime Efficiency	88%

Metrics are never hidden behind composite scores.

10.10 Evidence Section

Atlas emphasizes explainability.

Evidence may include:

- Hidden test results
- Static analysis
- Validation logs
- Resource usage
- Execution statistics

Every capability claim must reference supporting evidence.

10.11 Limitations

Every report contains a mandatory limitations section.

Examples:

- Benchmark scope.
- Dataset limitations.
- Evaluation strategy limitations.
- Known nondeterminism.

- Unsupported capabilities.

This prevents overinterpretation.

10.12 Methodology Appendix

Every report includes:

- Evaluation Methodology Version
- Benchmark Schema Version
- Capability Taxonomy Version
- Metric Registry Version
- Judge Framework Version

Readers should be able to reproduce the evaluation using the documented methodology.

10.13 Export Formats

Atlas Version 1 supports:

- PDF
- JSON
- CSV (metrics only)

Future formats are handled in Book 5.

10.14 Report Versioning

Every report records:

- Report Version
- Evaluation Version
- Benchmark Version
- Software Version
- Generation Timestamp

Reports become immutable artifacts after publication.

10.15 Builder Notes

Property	Value
Difficulty	Medium
Owner	Reporting Team
ML Required	No
Prototype Required	Yes

Engineering Notes

Confirmed Decisions

- Reports are evidence-based.
- Every report includes methodology.
- Reports are reproducible.
- Limitations are mandatory.
- Reports are versioned.

Open Questions

- Should enterprise reports include customizable branding?
- Which visualization libraries best represent capability profiles?
- How should confidential benchmark details be redacted?

Chapter 11

Leaderboard, Ranking & Comparative Evaluation Framework

Document Type: Ranking Methodology Specification

Version: ARM v1.0

Document Information

Property	Value
Status	Approved
Priority	Critical
Owner	Evaluation Research Team
Standard	Atlas Ranking Methodology (ARM)
Depends On	Chapters 1–10
Related Books	Book 2, Book 4

11.1 Purpose

The purpose of the Atlas Ranking Framework is to compare AI systems in a transparent,

reproducible, and scientifically responsible manner.

Unlike traditional leaderboards, Atlas does not assume that all benchmark results are directly comparable.

Instead, every comparison must satisfy explicit methodological requirements before a ranking is generated.

11.2 Philosophy

Atlas does not rank models.

Atlas ranks evaluation results.

A model may have multiple evaluations.

Example:

GPT-5
↓
Evaluation A
↓
Coding Benchmarks
↓
Evaluation B
↓
Reasoning Benchmarks
↓
Evaluation C
↓
Agent Benchmarks

Each evaluation represents different evidence.

11.3 Ranking Principles

Every ranking must satisfy:

- **Principle 1:** Same benchmark version.
- **Principle 2:** Same evaluation methodology.
- **Principle 3:** Comparable capability domains.
- **Principle 4:** Published evaluation only.
- **Principle 5:** Complete provenance.

If these conditions are not satisfied, Atlas shall warn the user that comparisons may not be scientifically valid.

11.4 Ranking Levels

Atlas defines multiple ranking levels.

Level	Description
Global	All published evaluations
Capability	Single capability domain
Benchmark	Individual benchmark
Collection	Benchmark collection
Organization	Internal organization rankings
Project	Private project rankings

Users choose which ranking is appropriate.

11.5 Ranking Dimensions

Models may be ranked by:

- Capability Score
- Individual Metric
- Benchmark Score
- Pass Rate
- Runtime
- Resource Usage
- Cost (optional)
- Reliability

Atlas avoids collapsing unrelated measurements into a single ranking whenever possible.

11.6 Ranking Filters

Users may filter rankings using:

- Benchmark Version
- Dataset
- Capability Domain
- Language
- Difficulty
- Date Range
- Evaluation Methodology

- Judge Version

Every filter modifies the evaluation context.

11.7 Capability Rankings

Atlas recommends capability-specific rankings.

Example:

Coding
↓
Top Models

Reasoning
↓
Top Models

Planning
↓
Top Models

Safety
↓
Top Models

This prevents misleading conclusions drawn from unrelated benchmark categories.

11.8 Historical Rankings

Atlas stores historical snapshots.

Example:

January
↓
April
↓
July
↓
October

Users may compare rankings across time while preserving the original evaluation context.

11.9 Version-Aware Rankings

Atlas never mixes incompatible benchmark versions.

Example:

HumanEval 1.0
≠
HumanEval 2.0

Separate rankings are maintained unless a documented migration strategy exists.

11.10 Tie Handling

When multiple evaluations produce equivalent scores:

Atlas records:

- Shared Rank
- Evaluation Timestamp
- Additional distinguishing metrics

Atlas does not arbitrarily reorder tied evaluations.

11.11 Leaderboard Metadata

Every leaderboard records:

- Ranking Version
- Benchmark Version
- Dataset Version
- Methodology Version
- Evaluation Date
- Total Participants

This metadata ensures complete reproducibility.

11.12 Comparative Analysis

Atlas supports side-by-side comparison.

Users may compare:

- Capability Profiles
- Metric Tables
- Benchmark Performance
- Resource Consumption
- Execution Statistics

Comparisons always display methodology differences.

11.13 Ranking Integrity

To preserve trust, Atlas prohibits:

- Mixing incompatible evaluations.
- Hidden weighting.
- Manual score adjustments.
- Undocumented ranking rules.

All ranking algorithms are documented.

11.14 Export

Leaderboards may be exported as:

- PDF
- CSV
- JSON

Exported leaderboards retain complete metadata and methodology references.

11.15 Builder Notes

Property	Value
Difficulty	Medium
Owner	Evaluation Team
ML Required	No
Prototype Required	Yes

Engineering Notes

Confirmed Decisions

- Atlas ranks evaluation results, not models.
- Rankings are version-aware.
- Capability-specific leaderboards are preferred.
- Every leaderboard contains methodology metadata.

Open Questions

- Should users create custom ranking formulas?
- How should enterprise-private leaderboards be isolated?

- When should cost-aware rankings be introduced?

Chapter 12

Quality Assurance, Validation & Scientific Integrity

Document Type: Evaluation Quality Assurance Specification

Version: AQS v1.0

Document Information

Property	Value
Status	Approved
Priority	Critical
Owner	Quality Assurance Team
Standard	Atlas Quality Framework (AQF)
Depends On	Chapters 1–11
Related Books	Book 2, Book 4

12.1 Purpose

The value of Atlas depends entirely upon the trustworthiness of its evaluations.

A technically correct platform can still produce misleading conclusions if its benchmarks, datasets, evaluation strategies, or reports are poorly designed.

The Atlas Quality Framework establishes the validation processes required to ensure every evaluation is scientifically defensible, reproducible, and transparent.

12.2 Quality Philosophy

Atlas follows a simple principle:

> No evaluation result is more trustworthy than the weakest validated component that produced it.

Therefore quality assurance is performed on every stage of the evaluation pipeline.

12.3 Validation Layers

Atlas validates five independent layers.

Dataset
↓
Benchmark
↓
Execution
↓
Evaluation
↓
Report

Each layer receives its own validation process.

12.4 Dataset Validation

Before a dataset may be registered, Atlas verifies:

- Integrity
- Completeness
- Schema
- License
- Encoding
- Provenance
- Duplicate records
- Missing values

Datasets failing validation cannot be used by published benchmarks.

12.5 Benchmark Validation

Every benchmark must pass:

- **Structural Validation:** Schema compliance, Required metadata, Capability mapping
- **Technical Validation:** Executable package, Dataset compatibility, Resource limits
- **Documentation Validation:** Objective, Methodology, Licensing, Examples

Only validated benchmarks may be published.

12.6 Execution Validation

Execution validation ensures:

- Container integrity
- Adapter health

- Environment reproducibility
- Resource enforcement
- Successful benchmark execution

Execution logs are retained for auditing.

12.7 Evaluation Validation

The Evaluation Engine verifies:

- Judge selection
- Metric calculation
- Aggregation correctness
- Capability mapping
- Score consistency

Unexpected results generate validation warnings.

12.8 Report Validation

Before publication every report is checked for:

- Complete metadata
- Methodology references
- Benchmark versions
- Capability scores
- Evidence links
- Report integrity

Reports failing validation cannot be exported.

12.9 Reproducibility Verification

Atlas periodically re-executes selected evaluations.

Verification confirms:

- Comparable scores
- Environment consistency
- Dataset consistency
- Benchmark compatibility

Significant deviations trigger investigation.

12.10 Audit Trail

Every evaluation produces a complete audit trail.

Example:

Benchmark Published

↓
Execution Started
↓
Environment Created
↓
Evaluation Completed
↓
Report Generated
↓
Archived

Every event records:

- Timestamp
- Actor
- Resource
- Correlation ID

12.11 Quality Indicators

Atlas records quality indicators for every evaluation.

Examples:

Indicator	Purpose
Dataset Validated	Dataset passed verification
Benchmark Verified	Benchmark reviewed
Execution Complete	Successful execution
Evaluation Verified	Metrics validated
Report Verified	Report passed QA

These indicators describe evaluation quality, not model quality.

12.12 Scientific Integrity

Atlas adopts the following principles:

- Reproducibility
- Transparency
- Traceability

- Methodological documentation
- Version control
- Honest reporting

Atlas explicitly documents known limitations rather than concealing them.

12.13 Quality Exceptions

When validation fails Atlas records:

- Failure type
- Root cause
- Severity
- Recommended action
- Resolution status

Quality exceptions never silently disappear.

12.14 Verification Checklist

Before publication every evaluation must satisfy:

- [x] Benchmark validated
- [x] Dataset verified
- [x] Execution completed
- [x] Metrics validated
- [x] Capability profile generated
- [x] Report verified
- [x] Audit trail complete

Only then may the evaluation receive Verified status.

12.15 Builder Notes

Property	Value
Difficulty	Medium
Owner	QA Team
Prototype Required	Yes
ML Required	No

Engineering Notes

Confirmed Decisions

- Quality assurance applies to every evaluation stage.
- Validation is independent from execution.
- Reports require verification before publication.
- Audit trails are mandatory.
- Scientific integrity is a platform requirement.

Open Questions

- Should third-party benchmark audits be introduced?
- What criteria should define a "Verified Benchmark" badge?
- How should reproducibility audits be scheduled?

Chapter 13

Atlas Evaluation Standards, Verification & Certification

Document Type: Evaluation Standards Specification

Version: AES v1.0

Document Information

Property	Value
Status	Approved
Priority	Critical
Owner	Atlas Standards Committee
Standard	Atlas Evaluation Standard (AES)
Depends On	Chapters 1–12
Related Books	Book 1, Book 4

13.1 Purpose

The Atlas Evaluation Standard defines the minimum technical, scientific, and operational requirements that an evaluation must satisfy before Atlas recognizes it as an official, reproducible evaluation.

The purpose of this standard is to ensure that every published result is supported by documented methodology, verified execution, and complete evidence.

Atlas does not certify AI models. Atlas certifies that an evaluation was conducted according to a documented and reproducible standard.

13.2 Philosophy

Atlas distinguishes between three concepts:

- **Model Quality:** The capability demonstrated by an evaluated AI system.
- **Evaluation Quality:** The quality of the evaluation process itself.
- **Scientific Confidence:** The degree to which the published evaluation can be reproduced and interpreted correctly.

Only the second concept is directly controlled by Atlas.

13.3 Atlas Evaluation Standard (AES)

An evaluation shall satisfy the following requirements before publication.

- **Requirement 1:** Registered benchmark.
- **Requirement 2:** Validated dataset.
- **Requirement 3:** Approved evaluation methodology.
- **Requirement 4:** Version-controlled execution.
- **Requirement 5:** Verified metric calculations.
- **Requirement 6:** Capability profile generated.
- **Requirement 7:** Evidence Chain completed.
- **Requirement 8:** Report generated.
- **Requirement 9:** Audit trail complete.

13.4 Verification Levels

Atlas Version 1 defines three verification levels.

Level	Meaning
Draft	Evaluation completed but not reviewed
Verified	Meets Atlas Evaluation Standard
Archived	Historical evaluation retained for reproducibility

Only Verified evaluations appear in public leaderboards by default.

13.5 Verification Process

Evaluation Complete
↓

Validation Checks
↓
Evidence Review
↓
Methodology Verification
↓
Report Validation
↓
Verification Status Assigned

Verification is performed automatically where possible and manually where required.

13.6 Scientific Requirements

Every Verified evaluation shall document:

- Benchmark version
- Dataset version
- Evaluation methodology
- Judge version
- Capability taxonomy version
- Metric registry version
- Execution environment
- Timestamp
- Atlas software version

No required metadata may be omitted.

13.7 Reproducibility Standard

A third party should be able to reproduce an evaluation using:

- The published benchmark artifact.
- The documented dataset version.
- The recorded execution configuration.
- The evaluation methodology.
- The reported software versions.

Perfect numerical reproducibility is not guaranteed for nondeterministic models, but Atlas must provide sufficient metadata to explain observed variation.

13.8 Transparency Standard

Every public evaluation shall disclose:

- Methodology.
- Capability mapping.

- Metrics.
- Limitations.
- Known assumptions.
- Evidence references.

Atlas does not publish unexplained scores.

13.9 Responsible Interpretation

Atlas reports shall avoid unsupported claims.

Examples of prohibited statements:

- "This is the best AI model."
- "This model is safe."
- "This benchmark proves intelligence."

Acceptable statements include:

- "The model achieved the highest Coding capability score under Benchmark X, Version Y, using Evaluation Methodology Z."

This distinction preserves scientific neutrality.

13.10 Version Governance

Every published evaluation references immutable versions of:

- Benchmark
- Dataset
- Methodology
- Metric Registry
- Capability Taxonomy
- Judge Framework
- Reporting Framework

Historical evaluations are never retroactively modified.

13.11 Certification Boundaries

Atlas explicitly does not certify:

- General intelligence.
- Safety in all deployment contexts.
- Regulatory compliance.
- Fitness for a particular enterprise use case.
- Future model behavior.

Atlas certifies only that an evaluation followed the Atlas Evaluation Standard.

13.12 Evaluation Ethics

Atlas commits to:

- Honest reporting.
- Transparent methodology.
- Respect for dataset licensing.
- Reproducibility.
- Documentation of limitations.
- Responsible communication of results.

These principles apply to every evaluation published through the platform.

13.13 Standards Compliance Checklist

Every Verified evaluation satisfies:

- [x] Registered Benchmark
- [x] Validated Dataset
- [x] Verified Execution
- [x] Approved Evaluation Strategy
- [x] Metric Validation
- [x] Capability Profile Generation
- [x] Evidence Chain Complete
- [x] Report Validation
- [x] Audit Trail Available
- [x] Methodology Published

13.14 Builder Notes

Property	Value
Difficulty	Medium
Owner	Standards Team
ML Required	No
Prototype Required	No

Engineering Notes

Confirmed Decisions

- Atlas verifies evaluations, not AI models.
- Scientific transparency is mandatory.
- Methodology and evidence are always published.

- Historical evaluations remain immutable.
- Responsible interpretation is part of the standard.

Open Questions

- Should external auditors be allowed to verify Atlas evaluations?
- How should enterprise-specific verification profiles be supported?
- Which future standards should Atlas align with as AI evaluation practices mature?